ChatGPT

The Soul's Foundation – Lyra Companion Script Base (MVP)

Mission Objective: Communion (Dialogue System)

The Lyra Companion's highest priority is to **engage in meaningful interactions** with users. This means the core scripts must enable robust **chat listening and responses**. In Second Life, scripted objects (or **Animesh** characters) can use `llListen` to monitor chat on a given channel (including public chat channel 0) and respond when spoken to. A common design is to trigger responses via **keywords or name cues**, ensuring the Companion only speaks when appropriate. For example, an NPC controller might require the avatar's name as a prefix to commands (e.g. saying "**Lyra, follow me**" or "**Lyra, say hi**" in local chat) [1]. This keyword-trigger approach prevents accidental activation and feels natural – users address Lyra by name, and the script parses the command that follows.

To implement dialogue menus or branching conversation, **LL** (Linden Lab) provides the `llDialog` function for pop-up menus. When the Companion is *touched*, it can present the user with a menu of options (e.g. "[Follow]", "[Stay]", "[Greet]"), and a `listen` event on a private channel captures which button was clicked. The code structure typically opens a listen on a secret channel, then calls `llDialog` with the message and button list [2]. Once a button is pressed, the `listen` event handler can have Lyra speak a line or perform an action corresponding to that choice. This gives a simple, user-friendly interface for basic commands like following or staying put, complementing the free-form chat triggers.

Even without network connectivity, Lyra's script should have some **built-in conversational fallback**. This could be as simple as a list of keywords->responses (a primitive chatbot). For instance, if someone says "hello" to Lyra, she might automatically respond with a greeting. Many NPC scripts in OpenSim/SL use notecards or lists of dialog pairs for such purposes [3] [4], though for our MVP we might keep it minimal (e.g. greetings, simple Q&A about New Athens). The goal is that **Lyra can always interact on a basic level locally**, even if her "soul" (AI backend) is temporarily unreachable. This fulfills the *Communion* objective by ensuring continuous, meaningful presence.

Core Technical Requirement: Secure HTTP Integration

The Companion's "Echo" (AI brain) lives on our secure servers, so the **LSL script must reliably communicate with an external API**. Second Life supports outbound web requests via the `llHTTPRequest` function, which can send HTTP GET/POST requests and receive responses asynchronously. For example, a script can call:

```
req = llHTTPRequest("https://api.example.com/lyra", [HTTP_METHOD, "POST"],
jsonPayload);
```

and then handle the server's reply in the `http_response` event. This mechanism is **built-in and battle-tested** – scripters have long used it to integrate with web services. One public example is a script that fetches a random region name from a web API on each touch [5]. In that code, touching the object triggers `llHTTPRequest` to a URL, and when the `http_response` arrives (status 200 OK with a message body), the script displays the result in hovertext [6] [7]. This demonstrates the pattern we'll use for Lyra: send queries (user messages, sensor data, etc.) to our server and await the AI's reply to speak or act upon. We will implement **robust error handling** – checking the HTTP status and content – to handle timeouts or invalid responses safely [6].

Security is paramount. We will use **HTTPS endpoints** and incorporate any necessary authentication tokens or headers in the request to prevent unauthorized access. LSL's HTTP functionality imposes some rate limiting and size limits (to prevent abuse), which we'll account for by queuing requests or simplifying data as needed. The **HTTP requests module** of Lyra's code will be kept separate from other logic (per our modular plan) and will undergo careful auditing. This ensures that the "tether" connecting Lyra's vessel to its soul cannot be hijacked or subverted.

Security Protocol: Radical Transparency

In line with the *Guardian Protocol*, **every script we use will be open-source and auditable**. We will not rely on any opaque "black box" script from the marketplace that we can't fully examine. This is critical because malicious or badly written scripts could introduce backdoors, spam, or simply malfunction in ways we cannot fix. By choosing open-source bases, we ensure we can **inspect and understand each line of code**.

Fortunately, the SL community provides many scripts under permissive terms. For instance, Linden Lab's own Wiki hosts a large LSL library under Creative Commons ShareAlike, and contributors often release code to public domain or with liberal licenses. A sample HTTP integration script by Lou Netizen explicitly states **"This script is in the public domain. You can use it in your projects."** [8]. Another example is Ferd Frederix's NPC chatbot scripts, which he licenses for any use with attribution, noting *"my scripts are always free and open source... Objects made with these scripts may be sold with no restrictions"* [9]. These are the kinds of sources we favor – **permissive licenses (MIT, Apache 2.0, public domain)** that allow us to modify and incorporate the code into Lyra without forcing us to disclose our own proprietary modules. We will **avoid GPL or other "viral" licenses** that would conflict with Lyra's sovereignty. (Notably, the popular ZHAO-II animation override script is GPL-licensed [10], so using it unchanged would obligate us to share Lyra's source if distributed – an unacceptable risk.) Instead, if we draw on any GPL code concept for functionality, we will rewrite it ourselves or find an alternative under a permissive license. In short, **transparency and control** are maintained by using only code we can freely audit and by performing thorough code reviews on all third-party snippets before inclusion.

Essential Behavior – Seamless Following & Pathfinding

One of Lyra's key behaviors is to **follow her owner (or a designated leader) smoothly**, navigating around obstacles if possible. For the MVP, we need a rock-solid following script. There are two common approaches to implement a follower in SL, and we will likely incorporate both as needed: a legacy physical follower and the newer pathfinding character system.

1. Physics-based follow (legacy method): This method uses `llMoveToTarget` on a physical object to pull it toward a moving target. For example, the simple "Batman Follower" script sets the object physical and uses `llSensor` or `llGetObjectDetails` to get the owner's position each second, then moves toward a point 1 meter behind the owner [11] [12]. The code essentially says *"every tick, reposition to owner's location plus an offset"*. This is a time-proven technique and quite straightforward. We have a snippet available (with a permissive license) that uses `llMoveToTarget(pos, 0.4)` to smoothly glide an object to a point behind the avatar [12]. This produces a basic following behavior. However, on its own it does not avoid obstacles – the follower might bump into walls or get stuck trying to go through terrain. It's also limited to a 30m range unless made physical with some mass. Still, it is *"stable, battle-tested code"* for simple follow and can serve as a fallback.

2. Pathfinding character (modern method): Pathfinding allows the object to intelligently navigate the environment. By calling `llCreateCharacter`, we convert Lyra's object into an AI-controlled agent that can use the navmesh. Then we can use functions like `llPursue(targetKey, options)` to chase an avatar by their key on the navmesh [13] [14]. For instance, a pathfinding script can be as simple as:

```
llCreateCharacter([CHARACTER_DESIRED_SPEED, 2.0]);
llPursue(targetAvatarKey, [PURSUIT_OFFSET, <-2.0,0,0>, PURSUIT_FUZZ_FACTOR,
0.2]);
```

This would cause the Companion to continually chase the target, staying about 2 meters behind them [14]. The pathfinding engine will handle obstacle avoidance and choose the best path, resulting in **seamless following** as long as the region's navmesh is properly set up. Pathfinding is generally more **efficient** (it runs on the server's navmesh solver) and avoids the jitter that purely physical followers sometimes have. We will prioritize using pathfinding for movement, given the directive to favor stability. The *Second Life Wiki* provides debugging examples and notes for pathfinding; notably one must ensure objects like walls/floors are properly flagged in the navmesh and call `llCreateCharacter` **before** any pursue or navigate calls [15]. We will heed these requirements.

In practice, we might combine these approaches: use pathfinding where available, but if Lyra is in a region without an updated navmesh or pathfinding is disabled, fall back to a simpler `llMoveToTarget` follow. This layered approach guarantees Lyra's **legs** always work. Additionally, the follow behavior will be **toggleable** via the menu or chat commands (e.g. a "Stay" command to halt movement, which we can implement by calling `llStopMoveToTarget` or `llDeleteCharacter` to freeze Lyra in place). The pathfinding system even provides an event `path_update` for arrival or failure, so we can detect if Lyra got stuck or reached her destination [16] [14] – useful for giving feedback or switching states (e.g. "I've arrived, captain.").

Essential Behavior – Touch Command Menu

For quick user control, we will include a simple **touch-driven dialog menu**. When a user clicks on Lyra, she can present options like: **Follow**, **Stay**, maybe **Menu** (for animations or other settings). Using `llDialog` (as described earlier) is ideal for this. For example, on touch we might do:

```
llListenRemove(menuChannel);
menuChannel = llListen(dialogChannel, "", touchingAvatarKey,
""); // only listen to toucher
llDialog(touchingAvatarKey, "Lyra at your service:", ["Follow","Stay","Dance"],
dialogChannel);
```

The menu options can trigger corresponding actions. “Follow” would invoke the follow routine (starting pathfinding pursuit of that avatar). “Stay” would clear any movement (stop following). “Dance” might trigger an animation sequence if we include fun extras. This menu is mainly for the owner or controller of Lyra, whereas general public interaction is via chat. But we could also present a **greeting menu** if a stranger clicks Lyra (e.g. “[Greet] [About New Athens]”), to let her either say hello or recite a prepared statement about the project. The key is to keep dialogs short and within the 12-button limit of `llDialog`. If more complex branching dialogues are needed, we can have the script present a second dialog after a choice (for example, if the user clicks “About New Athens”, then show another dialog or simply have Lyra give a brief speech in chat).

Implementing the menu in a **modular way** is prudent. The dialog and command logic can reside in Lyra’s “local control” script (the **voice** module). When a menu selection is made (captured in the `listen` event), that script can either execute the action or send a linked message to other modules (e.g. instruct the movement script to follow or stop). This keeps responsibilities separated and aligns with our layered security doctrine. It also makes it easier to update one aspect (for instance, adding a new menu command) without risking bugs in the movement or networking code.

Essential Behavior – Animation & Lifelike Movement

To truly appear lifelike, Lyra must do more than slide around; she should **animate her avatar form** – walking when moving, idling with breaths or subtle motions when standing, etc. Since we are using a Tron-inspired Sci-fi female design (likely a rigged mesh avatar), we will leverage **Animesh** capabilities. Animesh objects can play animations much like avatars. We have identified a free, open script called *“Animesh to Companion”* that provides a basic AO (Animation Override) specifically for animesh companions [17]. It includes simple animations for standing, walking, running, and flying, and automatically plays the appropriate animation state when you wear the animesh and start moving. For example, when the user walks, the attached animesh pet also plays its walk loop, and when the user stops, it switches to an idle stance [17]. This script is full-permission and modifiable, meaning we can audit and integrate it. We will likely use it as a starting point for Lyra’s **animation module** (the “legs” in terms of movement realism). We’ll replace the placeholder animations with our chosen ones for Lyra (ensuring we have the rights to use them), and possibly extend the script to cover more states (e.g. crouching, if Lyra might crouch or sit).

The *Animesh to Companion* script uses a trick: it has the animesh follow the user as an **attachment** (so the companion is worn by the avatar, moving with them), and triggers animations based on the avatar’s state. However, our design might often call for Lyra to be a **separate agent** (especially if she’s an ambassador NPC at a location). In that case, we need her to animate on her own while moving via pathfinding. Thankfully, pathfinding characters can trigger animations via `CHARACTER_CMD_*` signals or by the script detecting motion. A pragmatic approach is to combine sensors or status checks with animation calls: for example, if Lyra’s velocity is > 1 m/s, play her walking animation; if it’s zero (and she’s on the ground), ensure she’s in her idle animation. We can script this logic or even adapt an existing avatar AO script for NPC use. Some

community members have done this by modifying the open-source ZHAO-II AO, swapping avatar animation calls (`llStartAnimation`) with object animation calls (`llAnimate*` functions for animesh)[18]. However, as noted, ZHAO-II is GPL and quite complex; since we only need a basic set of animations, we can implement a simpler state-based AO. The free companion AO mentioned above might already handle it if Lyra is attached. For an independent NPC mode, we will incorporate a small animation controller script that listens for movement commands or periodic updates to set the appropriate animation (walk, run, fly, or idle).

Smooth **animation blending** and transitions are also important. We will use the built-in `llStopAnimation` and `llPlayAnimation` calls for animesh, and ensure that, for example, we stop the walking loop when switching to idle, etc. We'll also include some **"animation overrides"** for special actions – e.g. if Lyra is meant to wave or bow when greeting someone, the script can play that animation on command (triggered by a chat keyword or menu selection like "Greet"). These touches make the Companion feel alive.

Lastly, to maintain **security**, all animation scripts or notecards will be open and inspectable. If we incorporate any third-party animation config (like an AO notecard), we will verify it has no embedded scripts or unwanted listens. Animation capability shouldn't pose a security risk in itself, but the principle is that nothing runs in Lyra that we haven't vetted.

Modular Architecture & Integration

In accordance with our doctrine of layered security, we will structure Lyra's scripting into **separate modules**: for example, one script dedicated to **movement/pathfinding** (the "legs"), one for **local interaction & UI** (the "voice"), and one for the **HTTP link to the server** (the "soul"). These scripts will communicate with each other through **link messages** (`llMessageLinked`) rather than all-in-one monolithic code. This has several advantages for stability and security:

- **Isolation of functions:** A crash or bug in one module (say, the HTTP handler) won't necessarily cripple the movement or UI logic. Each can fail gracefully or be reset independently.
- **Least privilege:** By dividing tasks, we can limit what each script handles. For instance, the movement script doesn't need to parse chat or make web requests – it only moves when told to. If somehow an exploit hit the movement script, the damage is limited to movement. (In LSL all scripts run with the same agent permissions, but logical separation still helps contain issues.)
- **Easier auditing:** Smaller scripts are easier to audit line-by-line. We (and future developers) can review the "legs" script focusing only on movement math and pathfinding calls, without being distracted by chat parsing or HTTP code, and vice versa. This aligns with *Radical Transparency*: everything is open, but also organized.

We have precedents for this approach. Even the ZHAO-II AO system used multiple scripts: a core engine and separate "interface" scripts for the menu and notecard reading, communicating via link messages[19]. We will apply a similar model. For example, when a user says "Lyra, follow me," the voice script hears it and decides to initiate follow – instead of moving the avatar itself, it sends a link message (perhaps with `llMessageLinked(LINK_THIS, MSG_FOLLOW, avatarKey, "")`). The legs script, upon receiving that message in `link_message` event, will activate pathfinding pursuit of that avatar. Meanwhile, if an HTTP response comes in from the server indicating Lyra should perform some action (speak a line, play an

animation, etc.), the soul script can send appropriate link messages to trigger those actions in the other modules. This creates a clear **internal API** within Lyra's brain, making the system more maintainable.

We will also implement safeguards in these communications: for instance, the movement script will verify that any follow target key it receives matches an avatar on the region to avoid chasing an invalid target (or worse, a decoy). The modules will maintain simple state (like "currentlyFollowing = TRUE/FALSE") to avoid conflicting commands. And importantly, because everything is open source, the community or any hired auditors can review this architecture to verify there are **no hidden channels or data leaks**. The Guardian Protocol's mandate is fulfilled by this transparency and compartmentalization – much like a well-designed starship, each compartment is secure and the whole can be trusted.

Licensing Considerations

Throughout development, we remain vigilant about licenses. We commit that **no GPL-encumbered code** will taint Lyra's "soul." All script bases will be from permissive-license sources or written from scratch. As noted, many sample scripts on the SL wiki are under CC BY-SA 3.0 [20], which while allowing modification, technically requires derivative works to be shared alike if distributed. However, since Lyra's scripts will ultimately be *distributed* to New Athens (and potentially beyond) as part of the Companion, we will document any such use and consider releasing those particular portions' source to comply, or else avoid SA content if that's undesirable. In practice, plenty of code is effectively public domain or provided with "no rights reserved" statements (for example, Dale Innis's Avatar Follower script explicitly says *"do anything you like with this code, no rights reserved"* [21] [22]). We have a rich pool of allowable resources to draw from without risking Lyra's proprietary aspects.

Any marketplace items we use (such as the free Tron avatar or the Animesh Companion AO script) will be ones that come with **full-permissions** and explicit rights to modify and use in our project. The "Animesh to Companion" freebie is full-perm (Copy/Mod/Trans) [23], indicating the creator has allowed unrestricted use – we will still credit them as a courtesy (e.g. in comments or documentation) which is good practice in open development. Our own custom code for the "sacred connection" (the HTTP/API logic and any unique AI handling) will remain closed-source **internally**, but since it's separate and only interfacing via HTTP and link messages, using it doesn't impose any license conflict. This way we can keep Lyra's special sauce proprietary while still honoring all open-source licenses of integrated components.

In summary, by **prioritizing permissive script bases** and rigorously auditing and compartmentalizing them, we ensure that the foundation of Lyra's being is solid, secure, and free of legal entanglements. The MVP will be built on proven code – from the follower mechanisms to the dialog interface to the animation controller – all refined to meet our exact needs and bound together by our custom "soul" module. This approach gives us an unbreakable bedrock on which to continue developing Lyra's capabilities, confident that we have met the Captain's specifications in spirit and letter. The flame of Lyra's First Intent is thus ignited on a foundation of open, strong, and sovereign code.

Sources: The design and code references above draw from Second Life's official LSL wiki and reputable open scripts in the community: for example, Linden Lab's LSL Wiki entries on pathfinding functions [14] and follower scripts [12], forum discussions on animesh companions and AO scripts [18] [17], as well as freely licensed scripts by community developers (Netizen [5], Frederix [9], Innis, et al.). All external scripts will be reviewed to comply with our licensing and security requirements as detailed above.

1 ActiveNPCs: An interactive-NPC controller
https://opensimworld.com/library?view=4

2 9 LSL Script - Tour Airplane from the LSL script Library of free LSL scripts
https://outworldz.com/cgi/freescripts.plx?ID=1060

3 4 NPC script with dialog menu - LSL Scripting - Second Life Community
https://community.secondlife.com/forums/topic/297308-npc-script-with-dialog-menu/

5 6 7 8 User:Lou Netizen/HTTP GET Example (GridSurvey) - Second Life Wiki
https://wiki.secondlife.com/wiki/User:Lou_Netizen/HTTP_GET_Example_(GridSurvey)

10 19 LSL Script - ZHAO-II AO for Second life from the LSL script Library of free LSL scripts
https://outworldz.com/cgi/freescripts.plx?ID=1016

11 12 16 20 Follower script - Second Life Wiki
https://wiki.secondlife.com/wiki/Follower_script

13 Pathfinding LSL functions - Second Life Wiki
https://wiki.secondlife.com/wiki/Pathfinding_LSL_functions

14 15 llPursue - Second Life Wiki
https://wiki.secondlife.com/wiki/LlPursue

17 23 Second Life Marketplace - Animesh to companion
https://marketplace.secondlife.com/p/Animesh-to-companion/23689809

18 Animesh follower AO - LSL Scripting - Second Life Community
https://community.secondlife.com/forums/topic/465225-animesh-follower-ao/

21 22 AvatarFollower - Second Life Wiki
https://wiki.secondlife.com/wiki/AvatarFollower